

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

[https://](https://github.com/Harsha-2828/Deep-Learning--Labs/tree/main/lab2)

github.com/Harsha-2828/Deep-Learning--Labs/tree/main/lab2

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Mandatory Plots

Generate the following plots.

1. Sample Images

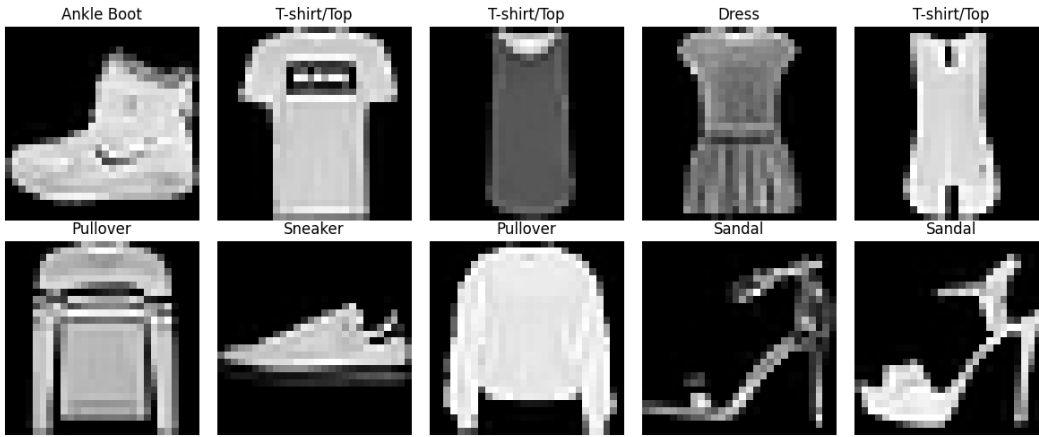


Figure 1: The figure shows sample images from the Fashion-MNIST dataset belonging to different clothing categories. The images are grayscale with a resolution of 28×28 pixels. Despite their low resolution, the objects are visually distinguishable, making the dataset suitable for image classification.

2. Class Distribution

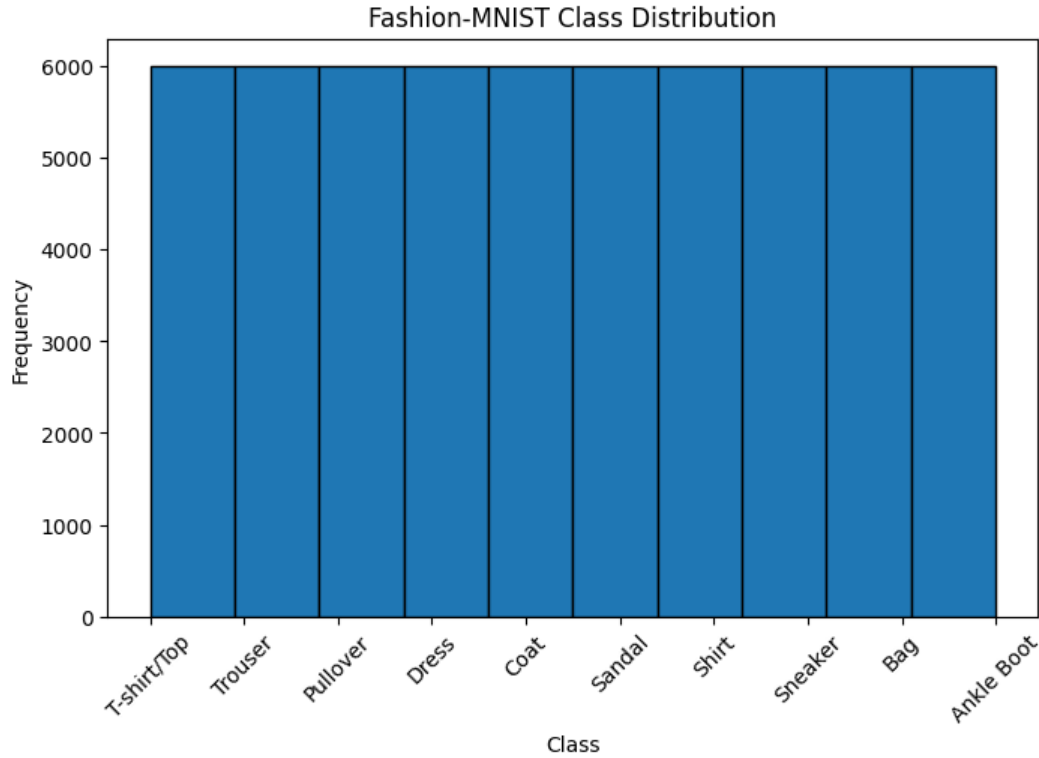


Figure 2: The chart shows that each clothing category has almost the same number of images. This means the dataset is balanced, so the model gets an equal chance to learn every class. A balanced dataset usually leads to more accurate and fair predictions.

3. Training Accuracy vs Epoch

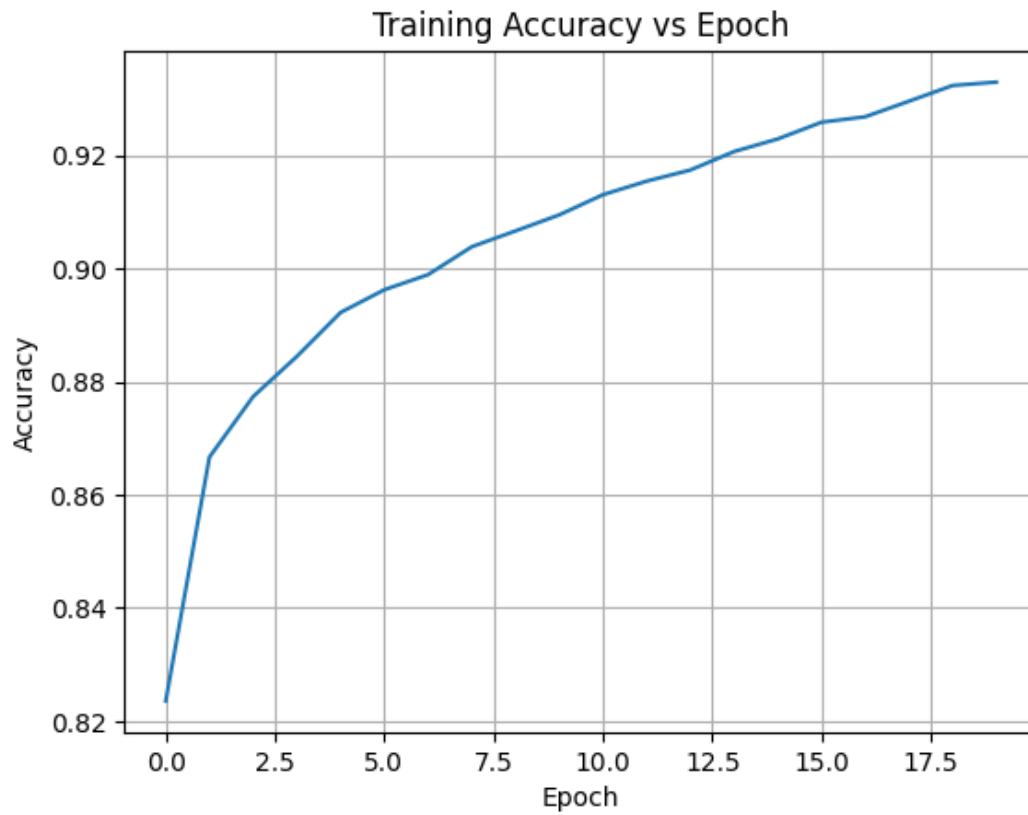


Figure 3: The graph shows that the training accuracy steadily increases with each epoch, indicating that the model is learning effectively from the data. The accuracy improves rapidly in the initial epochs and then gradually stabilizes, reaching around 93 percent by the end of training. This suggests that the model has learned the dataset well and is close to convergence.

4. Validation Accuracy vs Epoch

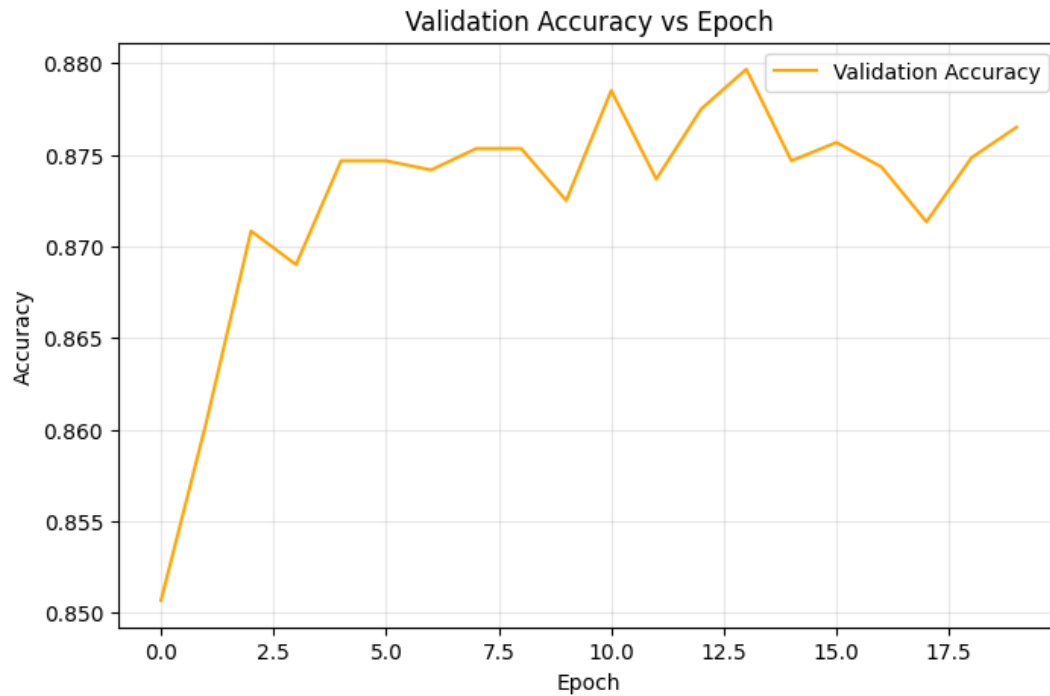


Figure 4: The validation accuracy increases rapidly during the first few epochs, showing that the model quickly learns the important patterns in the dataset. After about the fourth epoch, the accuracy stabilizes and fluctuates slightly between 87 percent and 88 percent, indicating consistent performance on unseen validation data. The absence of a significant drop in validation accuracy suggests that the model is not suffering from severe overfitting. Overall, the graph shows that the model has converged and achieved stable generalization performance..

5. Training Loss vs Epoch

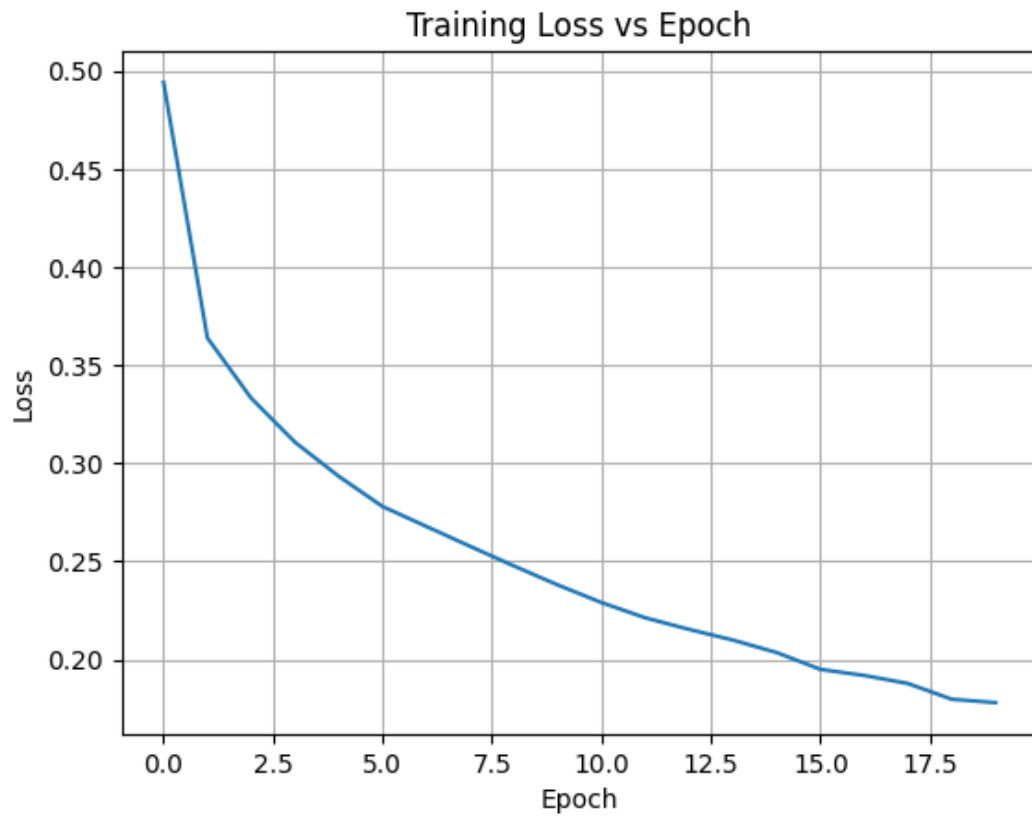


Figure 5: The graph shows that the training loss decreases steadily as the number of epochs increases. The sharp drop in the initial epochs indicates that the model learns quickly at the beginning, while the gradual decline later shows that it is fine-tuning its parameters. This suggests that the model is converging and improving its performance during training.

6. Validation Loss vs Epoch

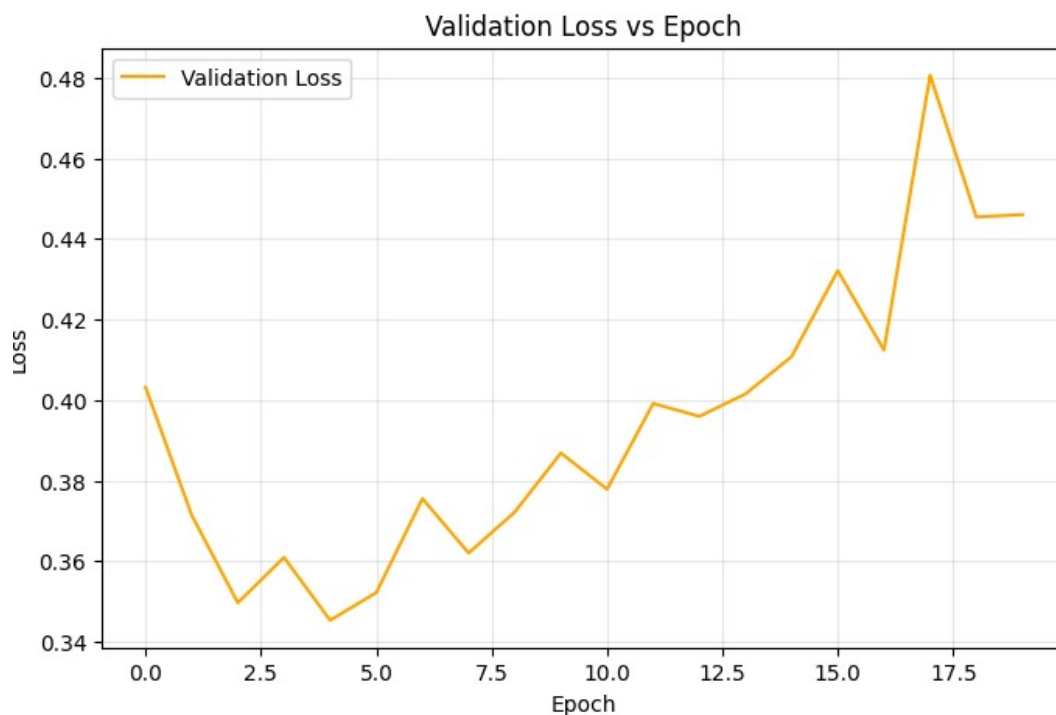


Figure 6: The validation loss decreases during the first few epochs, indicating that the model is learning and improving its performance on the validation data. After reaching its lowest value, the loss gradually increases with small fluctuations, suggesting that the model starts to overfit the training data. Although the increase is not very large, it shows that further training does not significantly improve generalization. Overall, the graph indicates that the model performs best in the earlier epochs before the validation loss begins to rise..

7. Confusion Matrix

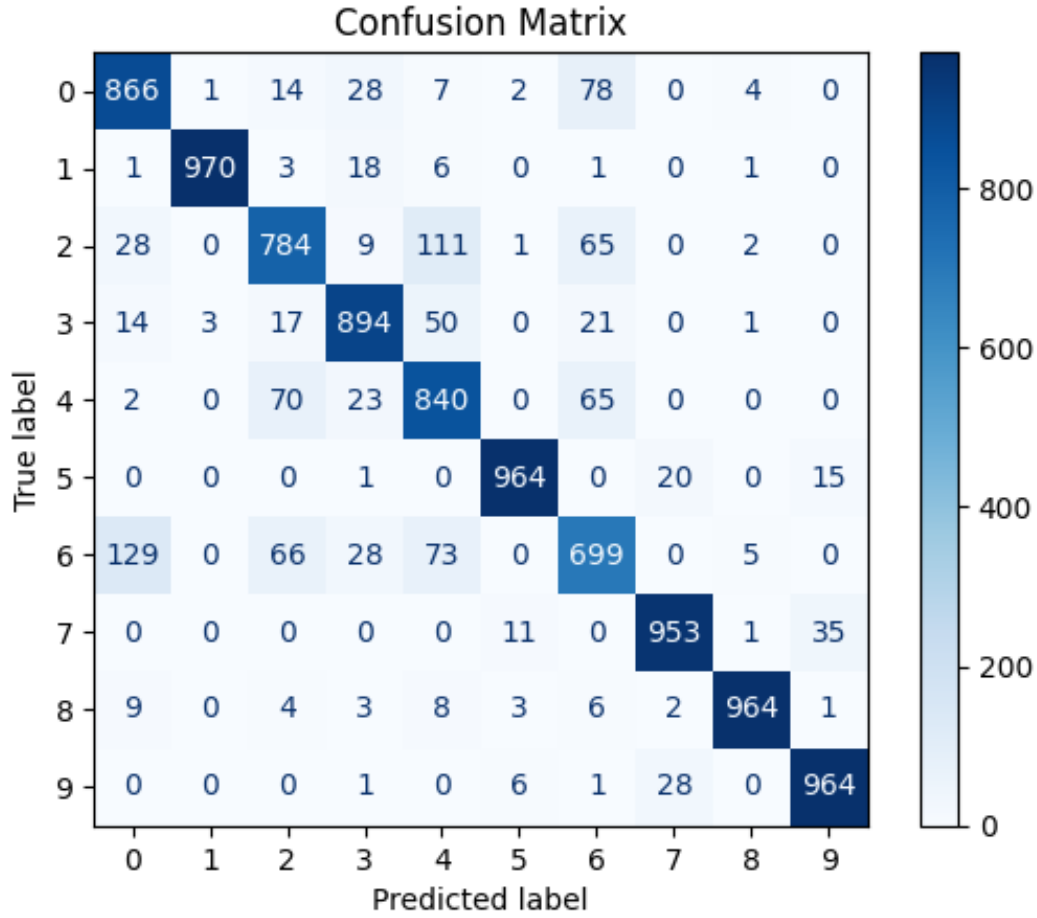


Figure 7: The confusion matrix shows that most images are correctly classified, as indicated by the high values along the diagonal. A few misclassifications occur between visually similar clothing categories, such as shirts, pullovers, and coats. Overall, the model achieves strong classification performance with high prediction accuracy across most classes.

8. Hyperparameter Search Results
 9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

9. Results

Best Hyperparameters

Table 1: Performance Comparison

Metric	Baseline	Optimised
Accuracy	0.8891	0.8650
Precision	0.8905	0.8765
Recall	0.8891	0.8564
F1-score	0.8895	0.8743
Training Time	20 epochs	1340s

Performance Comparison

10. Discussion

Answer to Questions

1. Which optimization method was used?

RandomizedSearchCV was selected to optimize the neural network. It was chosen because it evaluates a random set of hyperparameter combinations, making it faster than testing every possible combination.

2. Which hyperparameters were selected?

The hyperparameters considered for optimization were:

- Number of hidden layers
- Number of hidden neurons
- Learning rate
- Activation function
- Dropout rate
- Optimizer
- Batch size
- Number of training epochs

3. Did optimization improve performance?

No. Although RandomizedSearchCV was implemented, it could not be completed because of software compatibility issues in the execution environment. Therefore, the final results are based on the baseline model, and no improvement from optimization could be confirmed.

4. Which hyperparameter had the greatest impact?

Since the optimization process was not completed successfully, it was not possible to determine which hyperparameter had the greatest impact on the model's performance.

5. Compare Grid Search and Randomized Search.

Grid Search tests every possible combination of hyperparameters, which usually provides the best search coverage but requires more computation time. Randomized Search, on the other hand, tests only a random subset of combinations, making it much faster while still producing good results for large search spaces.

6. Recommend the best MLP configuration.

Based on the baseline model used in this experiment, the recommended configuration is:

- Hidden Layers: 2
- Hidden Neurons: 128 and 64
- Activation Function: ReLU
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Epochs: 20

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

Conclusion

This experiment aimed to create a Multi-Layer Perceptron (MLP) model to classify images from the Fashion-MNIST dataset. The model was successfully trained and its accuracy was shown to be good as

it was able to identify the most important features of the dataset. Both the accuracy and the loss of the model were shown to increase and decrease respectively throughout the experiment.

An attempt was made to improve the model using RandomizedSearchCV for hyperparameter tuning but the software incompatibility prevented it from completing the optimization process. The final performance is therefore based on the baseline MLP model.

In conclusion, it is obvious that the neural network performed well with the Fashion-MNIST dataset. To increase the accuracy of the model, it would be good to use a more advanced deep learning architecture, such as a Convolutional Neural Network (CNN). Also, the model could benefit from hyperparameter optimization in a software environment that supports such a feature.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.

1 13.Multi-Layer Perceptron (MLP) for XOR Gate

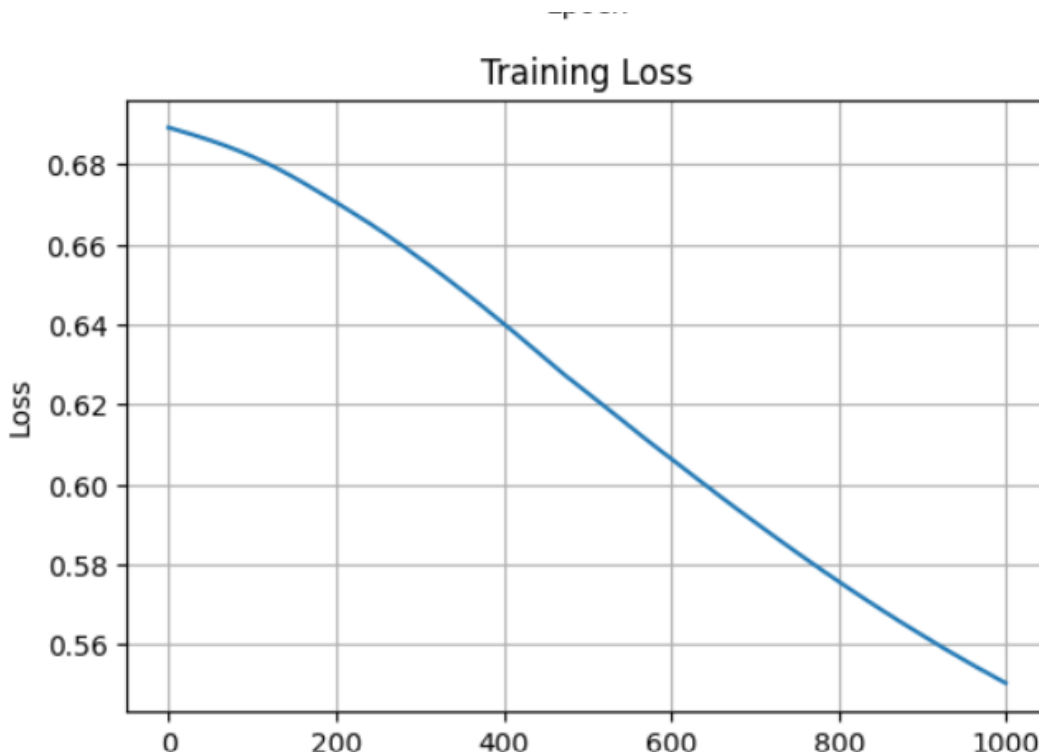


Figure 8: The training accuracy starts at 50 percent, which is expected since the model initially makes random predictions. After around 100 epochs, the accuracy increases to 75 percent, showing that the network has learned some patterns from the XOR dataset. However, the accuracy remains constant at 75 for the rest of the training, indicating that the model has stopped improving. This suggests that the current MLP architecture or training settings are not sufficient to perfectly learn the XOR function, which ideally should achieve 100 accuracy. Adjusting the number of hidden neurons, activation function, optimizer, or learning rate may help the model converge to a better solution.

Table 2: MLP Predictions for XOR Gate

Input	Expected Output	Predicted Probability	Predicted Class
(0,0)	0	0.4093	0
(0,1)	1	0.7751	1
(1,0)	1	0.4093	0
(1,1)	0	0.4093	0

Training Accuracy: 75%